

LEA Labs — Student Lab Assignment

Campus Tasks: Advanced State Management and Room Persistence

Module: Android Application Development — Project 1 Extension

Audience: Students who completed the in-memory Campus Tasks app

Estimated effort: 6–8 hours

Submission: Android Studio project, README, test evidence, and a 3–5 minute demonstration

Total: 100 marks

Challenge: Turn Campus Tasks from a temporary demo into a dependable offline-first app. When a learner closes, restarts, or rotates the app, their tasks should still be available and the screen should always explain what is happening.

1. Learning outcomes

After completing this lab, you should be able to explain the difference between transient events and persistent UI state; model a task as a Room entity; write a DAO with observable `Flow` queries and suspendable write operations; build a singleton Room database; place persistence behind a repository; expose screen state from a `ViewModel` with `StateFlow`; collect state safely in Jetpack Compose; and verify CRUD behavior, restart persistence, validation, loading, empty, and error states.

2. Scenario

The first Campus Tasks version stored tasks in a Kotlin list inside the screen. That version was useful for learning Compose, but it loses every task when the process ends. In this lab, you will keep the same user experience while replacing the temporary list with a local Room database.

The required user journey is:

1. The learner opens Campus Tasks and sees either an empty state or saved tasks.
2. The learner enters a title and optional note, then saves a task.
3. The new task appears without manually refreshing the screen.
4. The learner marks a task complete, edits it, or deletes it.
5. The learner closes and reopens the app; saved tasks remain.

6. Invalid input, loading, and database failures are communicated clearly.

3. Required architecture

Use the following one-way data path:

```
Compose UI
  | user events: add, edit, complete, delete, retry
  ▼
TaskViewModel — exposes TaskListUiState as StateFlow
  | delegates business operations
  ▼
TaskRepository
  | coordinates the data source
  ▼
TaskDao — Flow<List<TaskEntity>> + suspend writes
  ▼
RoomDatabase — SQLite-backed local persistence
```

The composable must not call the DAO directly. The ViewModel must not expose a mutable StateFlow. The UI should render from one screen-state object rather than maintaining a second competing copy of the task list.

4. Starting point

Begin from the Campus Tasks project created in Project Deck 2. Keep the existing visual identity, task-list interactions, and Compose screen where they are useful. You may refactor files, but your final project must be understandable to another student.

Recommended package structure:

```
app/  
  data/  
    local/TaskEntity.kt  
    local/TaskDao.kt  
    local/AppDatabase.kt  
    TaskRepository.kt  
  ui/task/TaskListUiState.kt  
  ui/task/TaskViewModel.kt  
  ui/task/TaskScreen.kt
```

5. Implementation tasks

Task A — Create the Room entity

Create a `TaskEntity` with at least these fields:

Field	Type	Requirement
<code>id</code>	<code>Long</code>	Auto-generated primary key
<code>title</code>	<code>String</code>	Required, trimmed before saving
<code>note</code>	<code>String</code>	Optional note with a sensible maximum length
<code>isCompleted</code>	<code>Boolean</code>	Defaults to <code>false</code>
<code>priority</code>	<code>Int</code> or enum mapping	Must support at least three levels, or explain a simpler design
<code>createdAt</code>	<code>Long</code>	Unix timestamp or equivalent

Example:

```
@Entity(tableName = "tasks")
data class TaskEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val title: String,
    val note: String = "",
    val isCompleted: Boolean = false,
    val priority: Int = 2,
    val createdAt: Long = System.currentTimeMillis()
)
```

Explain in your README why the database model is separate from the UI model if you introduce a separate `Task` or `TaskUiModel` class.

Task B — Create the DAO

The DAO must provide:

- an observable query returning `Flow<List<TaskEntity>>` ;
- insert or upsert support;
- update support for completion and edits;
- delete support;
- a query that returns tasks in a deliberate order, such as incomplete first and newest first.

Example:

```

@Dao
interface TaskDao {
    @Query("""
        SELECT * FROM tasks
        ORDER BY isCompleted ASC, priority ASC, createdAt DESC
        """)
    fun observeTasks(): Flow<List<TaskEntity>>

    @Insert
    suspend fun insert(task: TaskEntity): Long

    @Update
    suspend fun update(task: TaskEntity)

    @Delete
    suspend fun delete(task: TaskEntity)
}

```

Use parameterized Room queries. Do not build SQL by string concatenation.

Task C — Create the database singleton

Create an abstract `RoomDatabase` that exposes `TaskDao`. Construct it once for the application process. Do not create a new database inside every composable recomposition.

```

@Database(entities = [TaskEntity::class], version = 1, exportSchema = true)
abstract class AppDatabase : RoomDatabase() {
    abstract fun taskDao(): TaskDao

    companion object {
        @Volatile private var INSTANCE: AppDatabase? = null

        fun getInstance(context: Context): AppDatabase =
            INSTANCE ?: synchronized(this) {
                INSTANCE ?: Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "campus_tasks.db"
                ).build().also { INSTANCE = it }
            }
    }
}

```

Task D — Add the repository

The repository is the boundary between the rest of the app and the data source. It should expose the task stream and meaningful operations such as `addTask`, `toggleTask`, `updateTask`, and `deleteTask`. Keep the UI unaware of Room annotations and database construction.

```

class TaskRepository(private val dao: TaskDao) {
    val tasks: Flow<List<TaskEntity>> = dao.observeTasks()

    suspend fun addTask(title: String, note: String, priority: Int) {
        dao.insert(
            TaskEntity(
                title = title.trim(),
                note = note.trim(),
                priority = priority
            )
        )
    }

    suspend fun toggleTask(task: TaskEntity) {
        dao.update(task.copy(isCompleted = !task.isCompleted))
    }

    suspend fun deleteTask(task: TaskEntity) = dao.delete(task)
}

```

Task E — Define one screen UI state

Create a state object that can represent every important screen condition. At minimum include tasks, title input, note input, loading, error, and a validation message.

```

data class TaskListUiState(
    val tasks: List<TaskEntity> = emptyList(),
    val titleInput: String = "",
    val noteInput: String = "",
    val isLoading: Boolean = true,
    val errorMessage: String? = null,
    val validationMessage: String? = null,
    val isSaving: Boolean = false
)

```

The UI state is the output of state production. User events are inputs. For example, `TitleChanged("Read Kotlin")` is an event; the new `titleInput` value is state.

Task F — Build the ViewModel

The ViewModel owns mutable state, collects repository data, launches coroutines, validates input, and converts failures into user-visible state.


```

class TaskViewModel(
    private val repository: TaskRepository
) : ViewModel() {
    private val _uiState = MutableStateFlow(TaskListUiState())
    val uiState: StateFlow<TaskListUiState> = _uiState.asStateFlow()

    init {
        viewModelScope.launch {
            repository.tasks
                .catch { error →
                    _uiState.update {
                        it.copy(
                            isLoading = false,
                            errorMessage = error.message ?: "Could not load tasks"
                        )
                    }
                }
                .collect { tasks →
                    _uiState.update { it.copy(tasks = tasks, isLoading = false) }
                }
        }
    }

    fun onTitleChanged(value: String) {
        _uiState.update { it.copy(titleInput = value, validationMessage = null) }
    }

    fun saveTask() {
        val current = _uiState.value
        if (current.titleInput.trim().isEmpty()) {
            _uiState.update { it.copy(validationMessage = "Add a title before saving") }
            return
        }

        viewModelScope.launch {
            _uiState.update { it.copy(isSaving = true, errorMessage = null) }
            runCatching {
                repository.addTask(current.titleInput, current.noteInput, current.isCompleted)
            }.onSuccess {
                _uiState.update {
                    it.copy(titleInput = "", noteInput = "", isSaving = false)
                }
            }.onFailure { error →
                _uiState.update { it.copy(errorMessage = error.message) }
            }
        }
    }
}

```

```

        _uiState.update {
            it.copy(
                isSaving = false,
                errorMessage = error.message ?: "Could not save task"
            )
        }
    }
}

fun toggleTask(task: TaskEntity) {
    viewModelScope.launch { repository.toggleTask(task) }
}

fun deleteTask(task: TaskEntity) {
    viewModelScope.launch { repository.deleteTask(task) }
}
}

```

If you use a custom `ViewModelProvider.Factory`, explain why it is needed and demonstrate that the application still creates one repository and one database instance.

Task G — Connect Compose to the ViewModel

Collect state with lifecycle awareness and send events into the ViewModel. The screen should not own a second list of tasks.

```

@Composable
fun TaskScreen(viewModel: TaskViewModel) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()

    when {
        uiState.isLoading → LoadingState()
        uiState.errorMessage ≠ null → ErrorState(
            message = uiState.errorMessage,
            onRetry = { /* implement retry if your design supports it */ }
        )
        uiState.tasks.isEmpty() → EmptyState()
        else → TaskList(
            tasks = uiState.tasks,
            onToggle = viewModel::toggleTask,
            onDelete = viewModel::deleteTask
        )
    }

    TaskInput(
        title = uiState.titleInput,
        onTitleChange = viewModel::onTitleChanged,
        note = uiState.noteInput,
        onNoteChange = viewModel::onNoteChanged,
        validationMessage = uiState.validationMessage,
        isSaving = uiState.isSaving,
        onSave = viewModel::saveTask
    )
}

```

Task H — Prove persistence and quality

Your demonstration must prove that a task remains after the app process is stopped and reopened. Also demonstrate an empty database, invalid title, completed-task styling, delete, and at least one database or loading error strategy. If you cannot trigger a real database failure, explain the failure path and show a test or controlled fake repository.

6. Extension challenge

Choose one extension for distinction-level work:

- Add a search field whose query filters the Room `Flow` .

- Add category or priority filters without duplicating the source of truth.
- Add a detail/edit screen with Navigation Compose and a shared ViewModel scope.
- Add a schema version 2 migration, such as a `dueDate` column, and prove old data remains available.
- Add an import/export feature using a JSON file while preserving validation.

7. Submission checklist

Submit a zipped Android Studio project, a README, a 3–5 minute video or live demonstration, and screenshots or test output showing the required states. The README must include the architecture diagram, setup steps, database schema, known limitations, and a short reflection answering: “What is the difference between an event, UI state, and persisted data in my app?”

Do not submit generated code that you cannot explain. During review, you may be asked to trace one event from the button click to the database write and back to the rendered list.

8. Grading rubric — 100 marks

Criterion	Excellent	Developing	Limited	Marks
Room data model	Entity is well designed, validated, documented, and supports required task behavior.	Entity works but has weak defaults, naming, or documentation.	Entity is incomplete or not connected correctly.	15
DAO and queries	Observable Flow plus reliable CRUD operations, parameterized queries, and deliberate ordering.	Most operations work with minor query or ordering weaknesses.	Missing operations, unsafe query design, or no observable updates.	15
Database and repository	Singleton database is correctly constructed; repository cleanly hides data-source details.	Works but has lifecycle, construction, or separation concerns.	Database is repeatedly created or UI calls Room directly.	10
ViewModel and state design	Uses immutable exposed StateFlow, clear events, validation, loading, empty, error, and success states.	Basic state works but some states or event boundaries are unclear.	Mutable state is exposed or logic is scattered through the UI.	20
Compose integration	UI renders from one source of truth, collects lifecycle-aware state, and supports full CRUD interaction.	Main path works with small interaction or lifecycle gaps.	UI uses a competing local list or interactions are incomplete.	15
Persistence proof and testing	Restart persistence, CRUD, validation, empty/error states,	Persistence and core CRUD are shown but	No convincing restart proof or	15

Criterion	Excellent	Developing	Limited	Marks
	and meaningful test evidence are demonstrated.	evidence is incomplete.	testing evidence.	
Accessibility and UX	Clear labels, sensible focus/order, readable error states, accessible controls, and coherent empty/loading states.	Usable overall but misses some inclusive details.	Controls or feedback are confusing or inaccessible.	5
Code quality and explanation	Readable structure, sensible naming, README architecture explanation, and student can explain the data flow.	Code is understandable but lightly documented or unevenly organized.	Code is difficult to follow or cannot be explained.	5

Performance bands

- **85–100:** Distinction — reliable Room-backed app with clear architecture, strong evidence, and thoughtful extension.
- **70–84:** Merit — complete working solution with minor quality or documentation gaps.
- **50–69:** Pass — core persistence and interaction work, but architecture or evidence needs improvement.
- **0–49:** Resubmit — key requirements are missing, unverified, or not demonstrably understood.

9. Instructor facilitation guide

Begin by reproducing the original failure: add a task, kill the app, and reopen it. Ask students whether the task is UI state, business state, or persisted data. Then draw the one-way data path before showing code. During the lab, require a checkpoint after the entity and DAO, another after the ViewModel, and a final persistence demonstration. Common misconceptions to

address are treating Room as a UI state holder, creating a database in a composable, exposing `MutableStateFlow` , and assuming a successful insert automatically means the UI has a reliable loading/error experience.

References